



Contact for Further Details:

✉ codeazyclasses@gmail.com

☎ +91 93511 13029

Python String

Strings are sequence of characters written inside quotes. It can include letters, numbers, symbols and spaces. Python does not have a separate character type.

- A single character is treated as a string of length one.
- Strings are commonly used for text handling and manipulation.

Creating a String

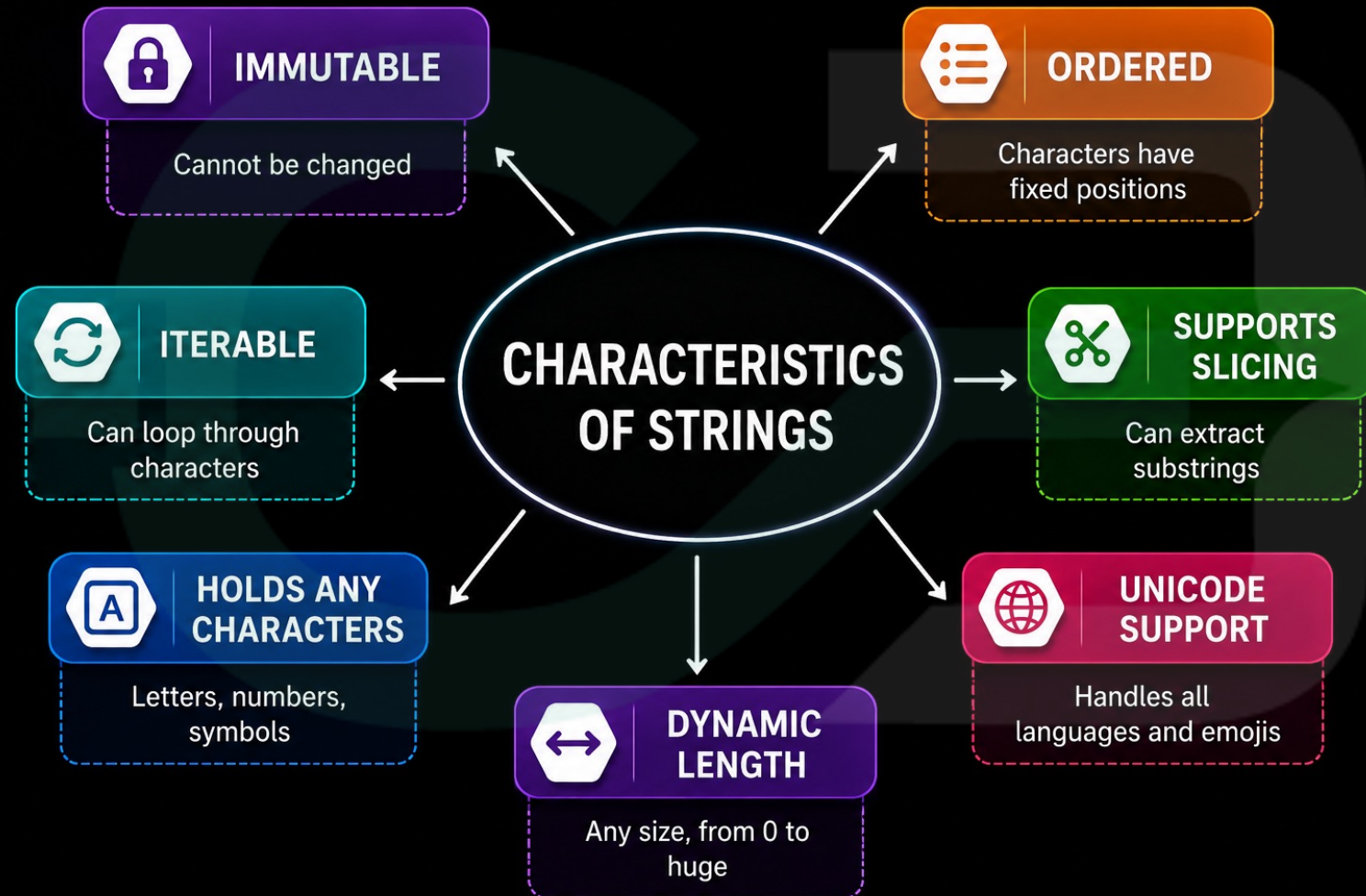
Strings can be created using either Single quotes ' '

- Double quotes " "
- Triple quotes '""' or """" (used for multiline string)

Example:

```
name = "Python"  
city = 'Jodhpur'  
message = """"Welcome to Python Programming"""
```

Characteristics of Python Strings



- **Immutable:** Once a string is created, we cannot change it. Any operation in terms of modifying a string will actually make a new string.
- **Ordered:** Strings are ordered collections of characters where each character has a fixed index (starting from 0). We can access the characters using their position.
- **Iterable:** We can iterate through each character of a string using Python loops, like for and while.
- **Support Slicing:** We can slice a string to extract substrings with the help of [start : end] syntax.
- **Unicode Support:** By default, strings in Python 3 are stored as Unicode. This allows us to handle international languages and emojis efficiently.
- **Dynamic Length:** We can initialize a string of any length, from an empty string to a string consisting of millions of characters.
- **Can contain any characters:** A string can include letters (A-Z, a-z), digits (0-9), special characters like @, #, \$, %, etc., spaces, tabs, and even newlines.

Accessing Characters in String

0	1	2	3	4	5	6
C	O	D	E	A	Z	Y
-7	-6	-5	-4	-3	-2	-1

Example:

1. `# python program to access characters in a string`
2. `# given string`
3. `s = "CODEAZY"`
4. `print("Given String:", s)`
5. `# accessing characters using indexing`
6. `print("s[0] =", s[0])`
7. `print("s[5] =", s[5])`

Accessing String with Negative Indexing

Example:

```
1. # python program to access characters from back of the string
2. # given string
3. s = "CODEAZY"
4. print("Given String:", s)
5. # accessing characters using negative indexing
6. print("s[-1] =", s[-1])
7. print("s[-6] =", s[-6])
```

String Slicing

The format for slicing a string is `string_name[start : end]`

Example:

1. `# python program to slice a string`
2. `# given string`
3. `s = "codeazy"`
4. `print("Given String:", s)`
5. `# getting characters from index 1 to 4`
6. `print("s[1:5] =", s[1:5])`
7. `print("s[:4] =", s[:4])`
8. `print("s[4:] =", s[4:])`
9. `print("s[::-1] =", s[::-1])`

String Function

S.No	String Functions	Description	Program	Output
1	len()	Returns total characters.	<pre>text = "Python" print(len(text))</pre>	6
2	max()	Returns largest character.	<pre>print(max("Python"))</pre>	y
3	min()	Returns smallest character.	<pre>print(min("Python"))</pre>	P
4	sorted()	Sorts characters.	<pre>print(sorted("python"))</pre>	['P', 'h', 'n', 'o', 't', 'y']

String Method

S.No	String Method	Description	Program	Output
1	upper()	Converts string into uppercase.	<pre>text = "python" print(text.upper())</pre>	PYTHON
2	lower()	Converts string into lowercase.	<pre>text = "PYTHON" print(text.lower())</pre>	python
3	capitalize()	Capitalizes first letter	<pre>text = "python programming" print(text.capitalize())</pre>	Python programming
4	title()	Capitalizes first letter of every word.	<pre>text = "python programming" print(text.title())</pre>	Python Programming
5	swapcase()	Changes uppercase to lowercase and vice versa.	<pre>text = "PyThOn" print(text.swapcase())</pre>	pYtHoN

String Method

6	casefold()	Used for aggressive lowercase comparison.	<pre>text = "HELLO" print(text.casefold())</pre>	hello
7	strip()	Removes spaces from both sides.	<pre>text = "Python" print(text.strip())</pre>	Python
8	lstrip()	Removes left spaces.	<pre>text = "Python" print(text.lstrip())</pre>	Python
9	rstrip()	Removes right spaces.	<pre>text = "Python" print(text.rstrip())</pre>	Python
10	replace()	Replaces old value with new value.	<pre>text = "I like Java" print(text.replace("Java", "Python"))</pre>	I like Python

String Method

11	split()	Converts string into list.	<pre>text = "Python Java C" print(text.split())</pre>	['Python', 'Java', 'C']
12	join()	Join list elements into string. join() is considered one of the most useful Python string methods for combining iterable strings.	<pre>data = ['Python', 'Java', 'C'] print("- ".join(data))</pre>	Python - Java - C
13	find()	Finds substring position. Or we can say that it finds the first occurrence of the specified value.	<pre>text = "Python" print(text.find("t"))</pre>	2 If not found: -1
14	index()	Same as find() but gives error if not found.	<pre>text = "Python" print(text.index("h"))</pre>	3
15	count()	Counts occurrences.	<pre>text = "Python Programming" print(text.count("m"))</pre>	2

String Method

16	startswith()	Checks starting word.	<pre>text = "Python" print(text.startswith("Py"))</pre>	True
17	endswith()	Checks ending word.	<pre>text = "Python" print(text.endswith("on"))</pre>	True
18	isalpha()	Checks alphabet only.	<pre>text = "Python" print(text.isalpha())</pre>	True
19	isdigit()	Checks digits only. Use isdigit() when you only want standard decimal digits, possibly in various scripts. Great for parsing IDs or zip codes.	<pre>text = "12345" print(text.isdigit())</pre>	True
20	isnumeric()	Checks numeric values. Use isnumeric() when you want to accept numeric forms like fractions or superscripts. Useful in math processing or display parsing.	<pre>text = "123" print(text.isnumeric())</pre>	True

String Method

21	isalnum()	Checks alphabets + numbers.	<pre>text = "Python123" print(text.isalnum())</pre>	True
22	islower()	Checks lowercase.	<pre>text = "python" print(text.islower())</pre>	True
23	isupper()	Checks uppercase.	<pre>text = "PYTHON" print(text.isupper())</pre>	True
24	istitle()	Checks title case.	<pre>text = "Python Programming" print(text.istitle())</pre>	True
25	center()	Aligns string at center.	<pre>text = "Python" print(text.center(10, '*'))</pre>	**Python**

String Method

26	zfill()	Adds zeros at beginning.	num = "25" print(num.zfill(5))	00025
27	partition()	Splits string into 3 parts.	text = "Python-Java" print(text.partition("-"))	('Python', '-', 'Java')
28	encode()	Converts string into encoded bytes.	text = "Python" print(text.encode())	b'Python'

String Operators

S.No	String Operators	Program	Output
1	CONCATENATION (+)	<pre>A = "HELLO" B = "WORLD" PRINT(A + " " + B)</pre>	HELLO WORLD
2	REPETITION (*)	<pre>PRINT("PYTHON " * 3)</pre>	PYTHON PYTHON PYTHON
3	MEMBERSHIP OPERATOR	<pre>TEXT = "PYTHON" PRINT("P" IN TEXT) PRINT("JAVA" NOT IN TEXT)</pre>	TRUE TRUE

Escape Character

Escape Character	Meaning
\n	New Line
\t	Tab Space
\\	Backslash
\'	Single Quote
\"	Double Quote

Example:

```
print("Hello\nWorld")
```